

MEDIATEK

MT8668 Android Audio User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	王江华	正式版

目录

版本记录	2
目录	3
图片目录	5
表格目录	5
1 概述	6
1.1 音频功能简单概述	6
1.1.1 名词解释	6
1.1.2 音频功能及框图	9
1.1.3 音频格式支持	12
2 音频软件架构	15
2.1 音频 HAL	16
2.1.1 音频 HAL 播放	16
2.1.2 音频 HAL 录制	17
2.2 ALSA 驱动架构概要	19
2.3 DAPM 及 DPCM 概述	19
3 AAOS 音量控制	21
3.1 在 XML 文件中配置音量	21
3.2 客制音量命令	22
4 外部硬件设备	23
4.1 MT8668 I2S 能力支持	23
4.2 I2S 连接应用简介	24
4.2.1 I2S 应用	25
5 联发科 Aurisys 和开放式 DSP	26
5.1 概述	26
5.1.1 Aurisys 结构	26
5.2 数据路径客制指南	27
6 Kernel DTS 配置	28
6.1 音频路径配置 (AFE HW)	30
6.2 音频路径配置 (INT ADSP)	32
6.2.1 XML 配置指令	33

6.2.2 mixer_target 和 mixer_source 33

6.2.3 任务属性 34

6.2.4 配置示例 35

7 音频 HAL 软件混音器.....37

7.1 音频 HAL 软件混音器代码架构..... 37

7.2 音频 HAL 软件混音器 XML 配置指南..... 38

7.2.1 先明确相关应用场景的实际需求..... 38

7.2.2 声明 BUS80/81/82 给 AudioPolicyManager 及 audioFlinger 39

7.2.3 配置 Audio HAL 的 Mixer 通路的路由并配置其为 Target..... 40

8 添加音频编 Codec 驱动42

9 总结.....44

附录 MTK MOL45

附件一 附加条款.....46

图片目录

图 2-1. MT8668 音频框图9

图 2-2. MT8668 音频框图详细信息10

图 2-3. MT8668 音频框图详细信息(接上图)11

图 3-1. 音频软件架构15

图 3-2. 音频软件数据流15

图 3-3. 音频 HAL 架构16

图 3-4. 播放架构17

图 3-5. 录制架构18

图 3-6. ALSA PCM 接口19

图 3-7. 音频前端和后端框图20

图 4-1. 音频音量控制21

图 4-2. 音频音量组配置21

图 4-3. 设备增益配置22

图 6-1. Aurisys 概念26

图 6-2. Aurisys 结构27

图 7-1. DTS 属性列表28

图 7-2. memif 定义29

图 7-3. 音频 HAL PCM Config 配置29

图 7-4. ADSP 架构30

图 7-5. 播放音乐的音频路径31

图 7-6. 将 Playback_3 传输到 PMIC32

图 7-7. ADSP 架构33

图 7-8. DSP 任务配置36

图 8-1. 音频 HAL 软件混音器软件架构37

图 8-2. 音频 HAL 软件混音器示意图38

表格目录

表 1-1.名词解释6

表 5-1. IP/Pin 映射表23

表 4-2. GPIO 功能24

表 4-3. I2S 接口分配建议25

1 概述

1.1 音频功能简单概述

在本文档中，我们将介绍音频功能，例如音量控制、音频框架、DSP 音频配置等。首先，我们将概述所支持的功能和音频框图。然后，我们将介绍 AAOS 音量控制及相关配置。最后，我们将介绍 Aurisys 音效框架和 DSP 音频 mix 通路配置。

1.1.1 名词解释

表 1-1.名词解释

缩写	释义
A2DP	Advanced Audio Distribution Profile 高级音频分配配置文件
AAC	Advanced Audio Coding 高级音频编码
ACF	Acoustic Correction Filter 声学校正滤波器
ADC	Analog-to-Digital Converter 模拟数字转换器
ADDA	Conversion from analog to digital and from digital to analog 模拟到数字和数字到模拟的转换
ADSP	Audio Digital Signal Processor 音频数字信号处理器
AEC	Acoustic Echo Cancellation 声回波消除
AFE	Audio Front End 音频前端
ALSA	Advanced Linux Sound Architecture 高级 Linux 声音架构
AMR	Adaptive Multi-Rate 自适应多速率
AOSP	Android Open Source Project 安卓开放源码项目
API	Application Programming Interface 应用程序接口
ARM	Advanced RISC Machines 高级精简指令集计算机
ASR	Acoustic Speech Recognition 声学语音识别
AWB	Audio Write Back 音频回写
BCK	Bit-Clock 位时钟
BE	Back End 后端
BLE	Bluetooth Low Energy 蓝牙低功耗
BT	Bluetooth 蓝牙

缩写	释义
CBR	Constant Bit Rate 固定比特率
CCCI	Cross Cores Communication Interface 跨核心通信接口
CPU	Central Processing Unit 中央处理器
CTS	Compatibility Test Suite 兼容性测试套件
CVSD	Continuously Variable Slope Delta 连续可变斜率增量调制
DAC	Digital-to-Analog Converter 数字模拟转换器
DAI	Digital Audio Interface 数字音频接口
DAPM	Dynamic Audio Power Management 动态音频功率管理
dB	Decibel 分贝
DC	Direct Current 直流电
DL	Downlink 下行链路
DMA	Direct Memory Access 直接内存访问
DPCM	Dynamic Pulse Code Modulation 动态脉冲编码调制
DRC	Dynamic Range Control 动态范围控制
DSP	Digital Signal Processor 数字信号处理器
EM	Engineer Mode 工程模式
EMI	External Memory Interface 外部内存接口
eTDM	Electrical Time Division Multiplexing 电时分复用
FAQ	Frequently Asked Questions 常见问题解答
FE	Front End 前端
FFT	Fast Fourier Transform 快速傅里叶变换
FIR	Finite Impulse Response 有限脉冲响应
FM	Frequency Modulation Broadcast 调频广播
GUI	Graphical User Interface 图形用户界面
HAL	Hardware Abstraction Layer 硬件抽象层
HCF	Headphone Compensation Filter 耳机补偿滤波器
Hz	Hertz 赫兹
I2S	Inter-IC Sound 集成电路间声音
IC	Integrated Circuit 集成电路
IIR	Infinite Impulse Response Filter 无限脉冲响应滤波器

缩写	释义
IP	Intellectual Property 知识产权
kbps	Kilobits per Second 千比特每秒
KHz	Kilohertz 千赫兹
LPF	Low-Pass Filter 低通滤波器
MD	Modem 调制解调器
MIDI	Music Instrument Digital Interface 乐器数字接口
MM	Multimedia 多媒体
NB	Narrowband 窄带
NVRAM	Non-Volatile Random Access Memory 非易失性随机存取存储器
OS	Operating System 操作系统
PA	Power Amplifier 功率放大器
PC	Personal Computer 个人计算机
PCM	Pulse Code Modulation 脉冲编码调制
PGA	Programmable Gain Amplifier 可编程增益放大器
PMIC	Power Management Integrated Chip 电源管理集成芯片
RISC	Reduced Instruction Set Computer 精简指令集计算机
RW	Read and Write 读和写
RX	Receive 接收
SCO	Synchronous Connection Oriented 同步连接导向
SD	Secure Digital 安全数字
SMP	Symmetric Multi-Processing 对称多处理
TX	Transmit 发送
UI	User Interface 用户界面
UL	Uplink 上行链路
USB	Universal Serial Bus 通用串行总线
VBR	Variable Bitrate 可变比特率
VoIP	Voice over Internet Protocol 网络语音协议
XML	Extensible Markup Language 可扩展标记语言

1.1.2 音频功能及框图

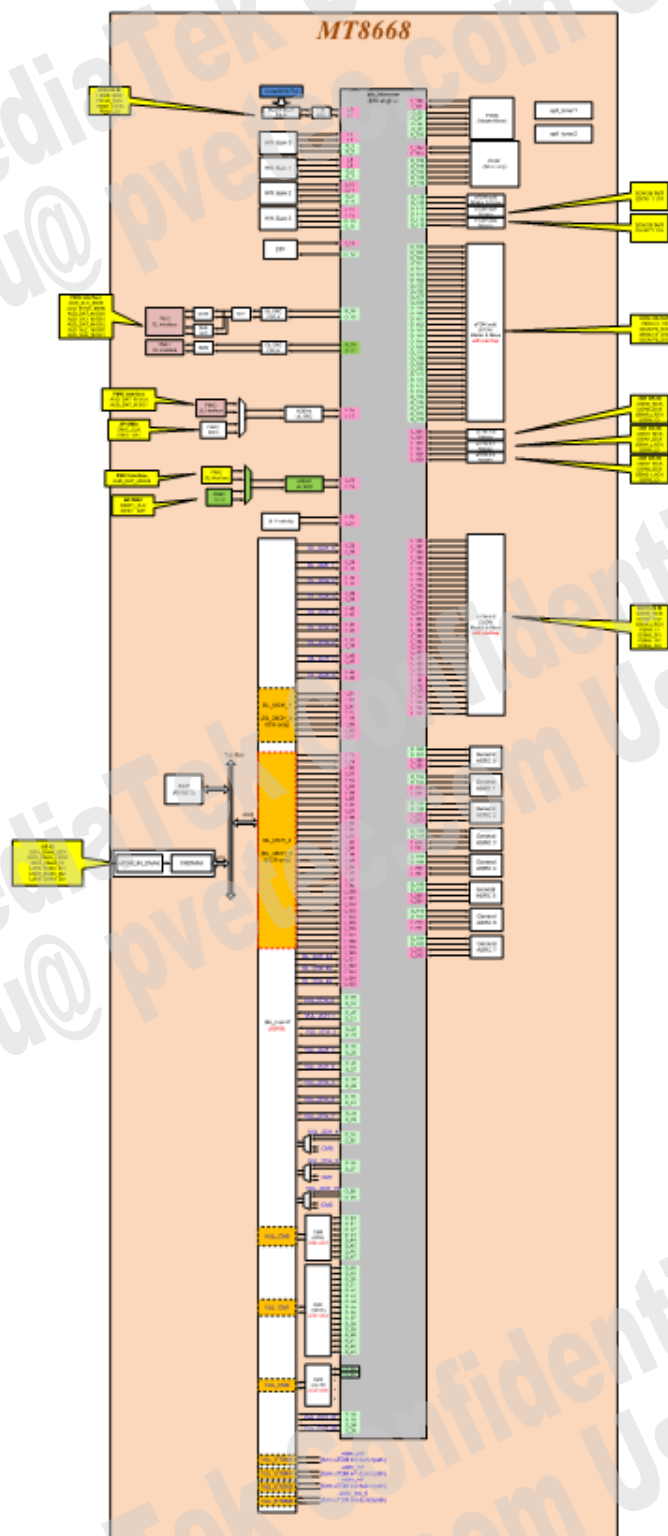


图 1-1. MT8668 音频框图

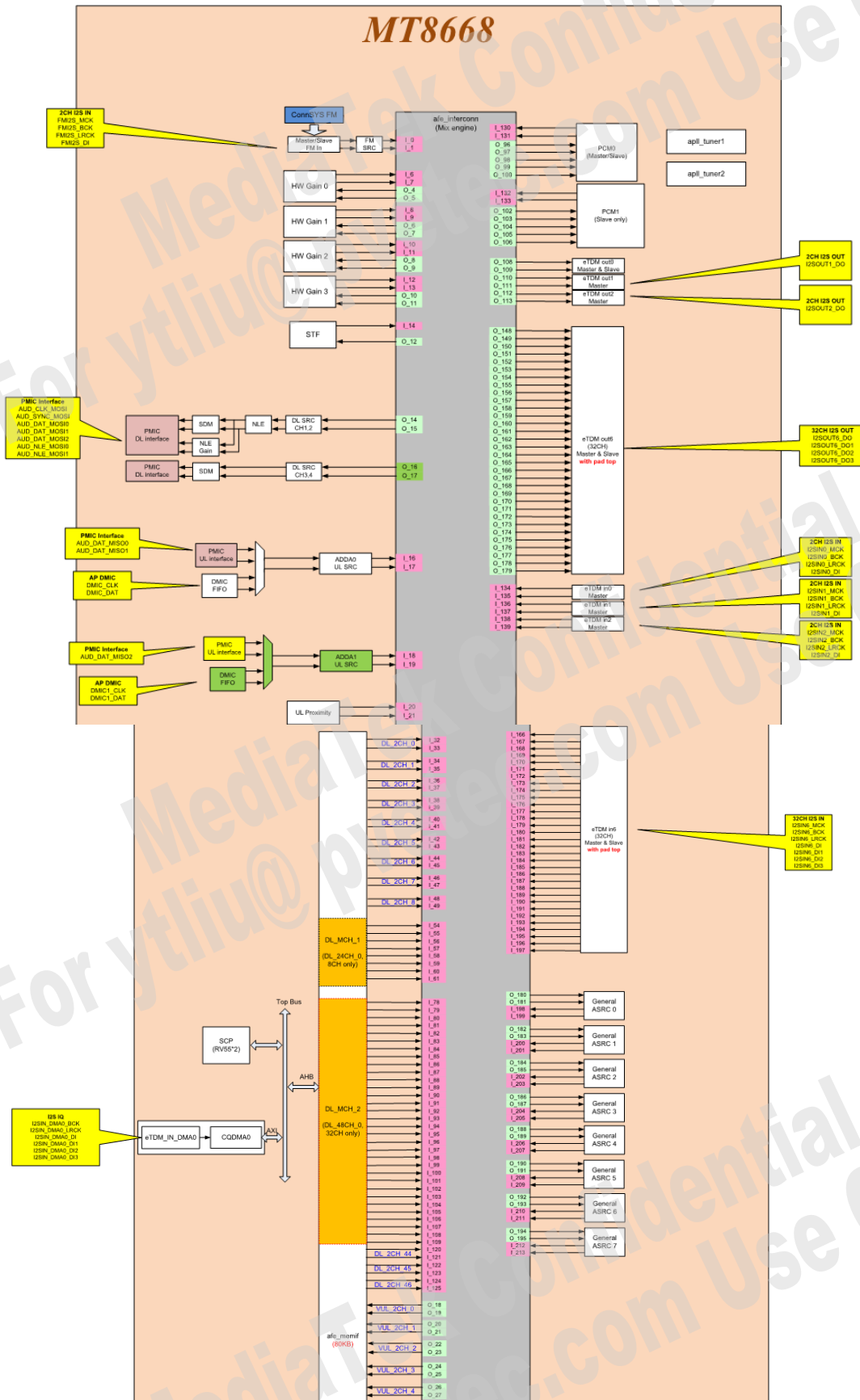


图 1-2. MT8668 音频框图详细信息

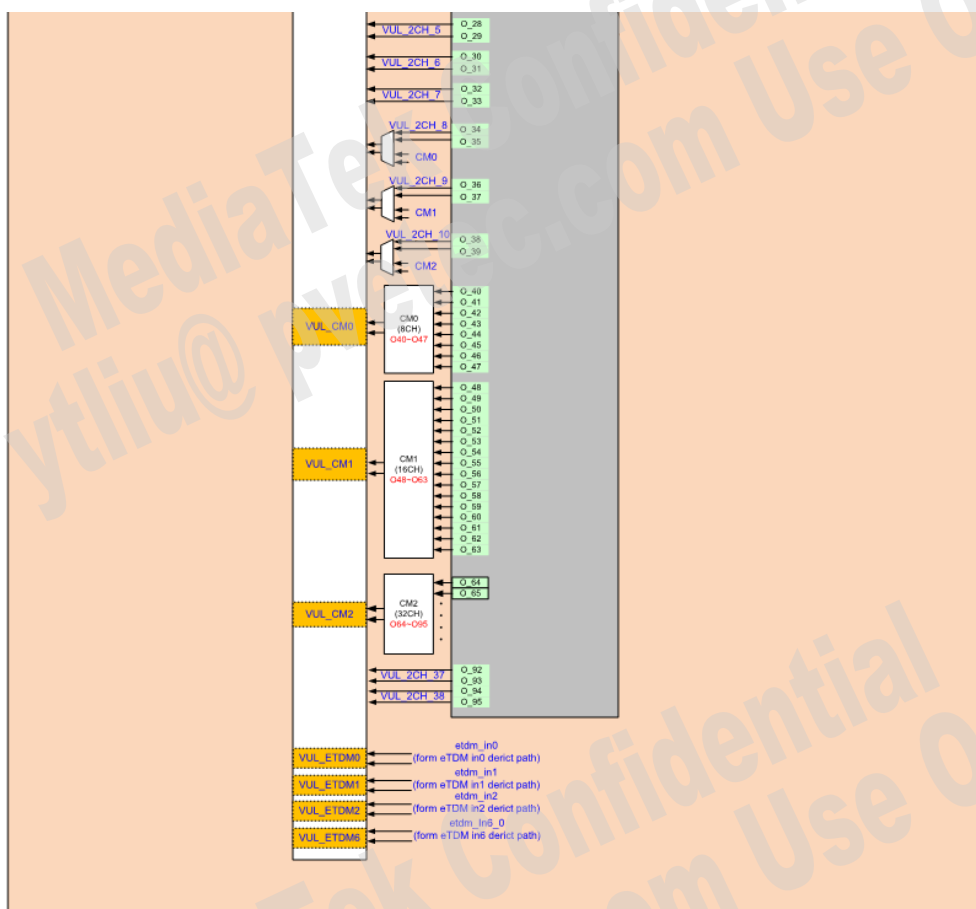


图 1-3. MT8668 音频框图详细信息(接上图)

- MT8668
 - 内部高分辨率、高灵活性的 AFE 互连中的 HW 增益
 - 8 组 Inter IC 音频接口 (I2S)
 - 主输出 *2 (2 通道/32 位/384K 采样率)
 - 主输出 *1 (eTDM, 32 通道/32 位/32K 采样率)
 - 主输入 *3 (2 通道/32 位/192K 采样率)
 - 主输入 *1 (32 通道/32 位/48K 采样率)
 - 从输入高速 I2S IQ *1 (DMA0: 8 通道/18 位/2048K 采样率)
 - FM 从输入(带 SRC)
 - 通用异步采样率转换器 *8(16 通道)
 - 主模式支持 8, 11.025, 12, 16, 22.05, 24, 32, 44.1, 48, 88, 96, 176 和 192kHz 采样率
 - 从模式支持 8, 11.025, 12, 16, 22.05, 24, 32, 44.1 和 48kHz 采样率
 - 支持 16/32 位总线宽度
 - Philip 标准和左对齐

- 2 组脉冲编码调制 (PCM)接口
 - Slave PCM for internal Modem*1
 - 支持 16/24 位立体声数据格式
- 音频编解码器
 - MP3, AAC, AAC+, AMR-NB, AMR-WB, OGG, WAV
- 第三方支持 (需要客户与第三方联系)
 - Dolby mobile

1.1.3 音频格式支持

- 播放

- AAC/HE-AAC v1/HE-AAC v2
 - Android Orientated Support
 - 8 kHz ~ 96 kHz; 8 kbps ~ 320 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR/CBR
 - File Extension: .aac (ADTS, ADIF), .m4a, .mp4, .3gp, .ts
 - Profile: 1) LC, HEAAC V1, V2; 2) LD, ELD
 - LC, 48 kHz, 128 kbps, stereo; HEv1, 22.05 kHz, 128 kbps, stereo; HEv2, 22.05 kHz, 32 kbps, stereo; ELD, 44.1 kHz, 128kbps
- AMR
 - Android Orientated Support
 - 8 kHz, 4.75 kbps~12.2 kbps
 - Mono Support
 - Bitrate mode: CBR
 - File Extension: .amr
 - 8 kHz, 12.2 kbps, mono
- AWB
 - Android Orientated Support
 - 16 kHz, 6.6 kbps~23.85 kbps
 - Mono Support
 - Bitrate mode: CBR
 - File Extension: .awb
 - 16 kHz, 23.85 kbps, mono
- MIDI
 - Android Orientated Support
 - 22.05 kHz
 - Stereo Support
 - File Extension: .mid, .midi, .smf, .rtttl, .xmf, .rtx, .ota, .imy
- MP3
 - Android Orientated Support
 - 8 kHz ~ 48 kHz; 8 kbps ~ 320 kbps

- Mono/Stereo Support
- Bitrate Mode: VBR/CBR
- File Extension: .mp3
- Profile: MPEG1-Layer3, MPEG2-Layer3, MPEG2.5-Layer3
- MPEG1, 48 kHz, 256 kbps, stereo
- OGG VORBIS
 - Android Orientated Support
 - 8 kHz ~ 192 kHz; 10 kbps ~ 320 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR
 - File Extension: .ogg, .oga
 - 48kHz, 250bps, stereo
- WAV (Raw)
 - Android Orientated Support
 - 6 kHz ~ 96 kHz; 48 kbps ~ 3072 kbps
 - 1 channel ~ 8 channels support
 - File Extension: .wav
- FLAC
 - Android Orientated Support
 - 8 kHz ~ 48 kHz; 87 kbps ~ 396 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR
 - File Extension: .flac
 - 44.1 kHz, 745kbps, level5, stereo
- 录制
 - AAC
 - Android Orientated Support
 - 8 kHz ~ 48 kHz; 8kbps ~ 160kbps
 - Mono/Stereo Support
 - Bitrate Mode: CBR
 - File Extension: .3gp, .aac
 - Profile: Low Complexity; High Efficiency; Enhanced Latency Delay
 - 48 kHz, 128 kbps, stereo (Low Complexity); 48 kHz, 128 kbps, stereo (High Efficiency); 48 kHz, 128 kbps, stereo (Low Latency Delay)
 - AMR
 - Android Orientated Support
 - 8 kHz; 4.75 kbps ~ 12.2 kbps
 - Mono Support
 - Bitrate Mode: CBR
 - File Extension: .3gp, .amr
 - 8 kHz, 12.2 kbps, mono
 - AWB
 - Android Orientated Support

- 16 kHz; 6.60 kbps~23.85 kbps
- Mono Support
- Bitrate Mode: CBR
- File Extension: .3gp, .awb
- 16 kHz, 23.85 kbps, mono
- FLAC
 - Android Orientated Support
 - 8 kHz ~ 48 kHz; 87 kbps ~ 396 kbps
 - Mono/Stereo Support
 - Bitrate Mode: VBR
 - File Extension: .flac
 - 44.1 kHz, 745kbps, level5, stereo

2 音频软件架构

图 2-1 和图 2-2 分别是音频软件架构和数据流。软件组件和数据路径可以在这两张图中看到。

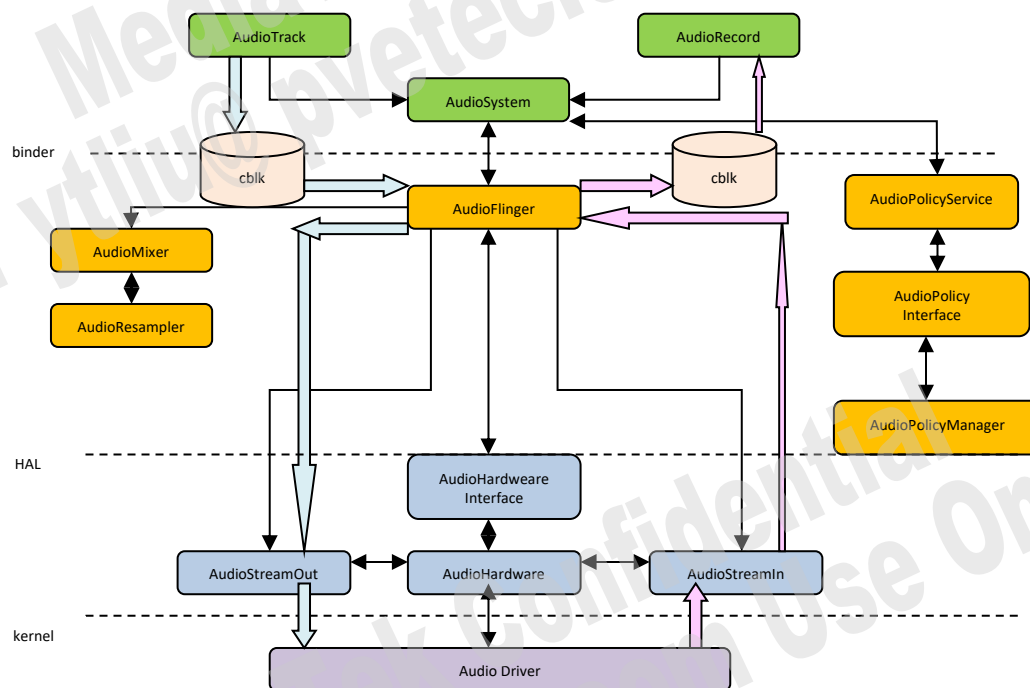


图 2-1. 音频软件架构

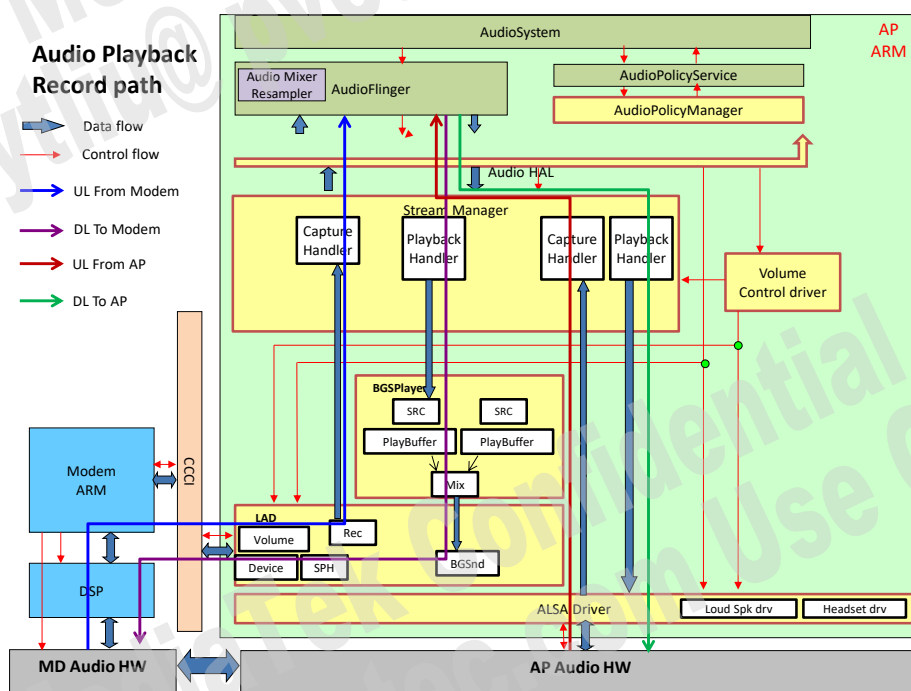


图 2-2. 音频软件数据流

2.1 音频 HAL

图 2-3 是音频 HAL 架构。

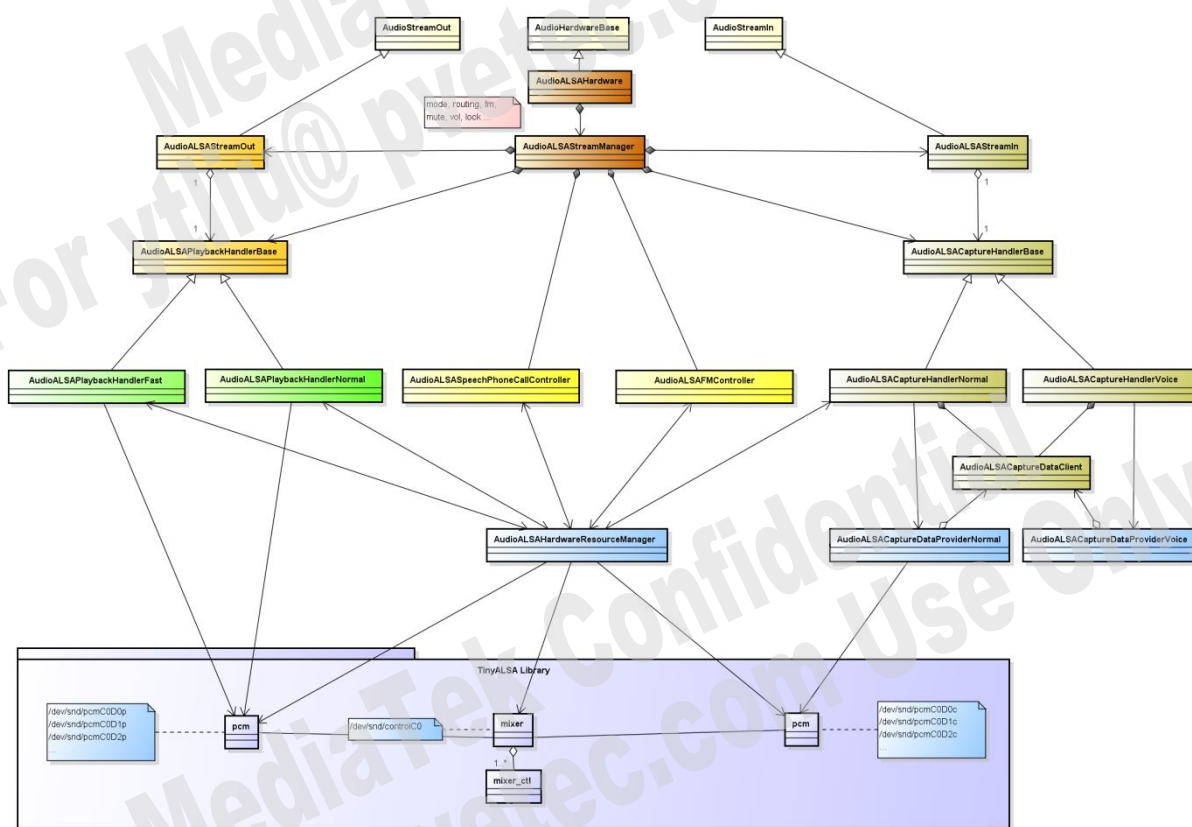


图 2-3. 音频 HAL 架构

HAL 的顶层控制接口是 AudioALSAHardware、AudioALSAStreamOut 和 AudioALSAStreamIn。这些类都使用 AudioALSAStreamManager 来控制音频模式、打开/关闭，输入/输出流。因此 AudioALSAStreamManager 将拥有所有音频环境信息，如模式、路由、音量和静音信息。

2.1.1 音频 HAL 播放

AudioALSAStreamOut: 执行写入/待机/路由操作的类

- 不实现细节本身，而是使用 Playback Handler 执行实际打开/关闭/路由/写入操作
- 不同场景使用不同类型的 Playback Handler
- 第一次写入时，Stream Out 将调用 StreamManager 中的 createPlaybackHandler() 来获取 Playback Handler 的指针
- 在 standby() 时，destroyPlaybackHandler()

不要在 playback handler 中设置硬件寄存器，而是使用 TinyALSA 库来调用

- `pcm_open()/pcm_close()`
 - 控制 AFE 硬件路径和内存设置
- `mixer_open()/mixer_close()`
 - 获取各种类型的混音器控制, 比如 `Speaker_Amp_Switch`, `Voice_Amp_Switch`, `Audio_Amp_R_Switch`, and `Audio_Amp_L_Switch`, 以控制编解码器驱动程序。
- `pcm_write()`
 - 将 PCM 数据写入 SRAM/DRAM

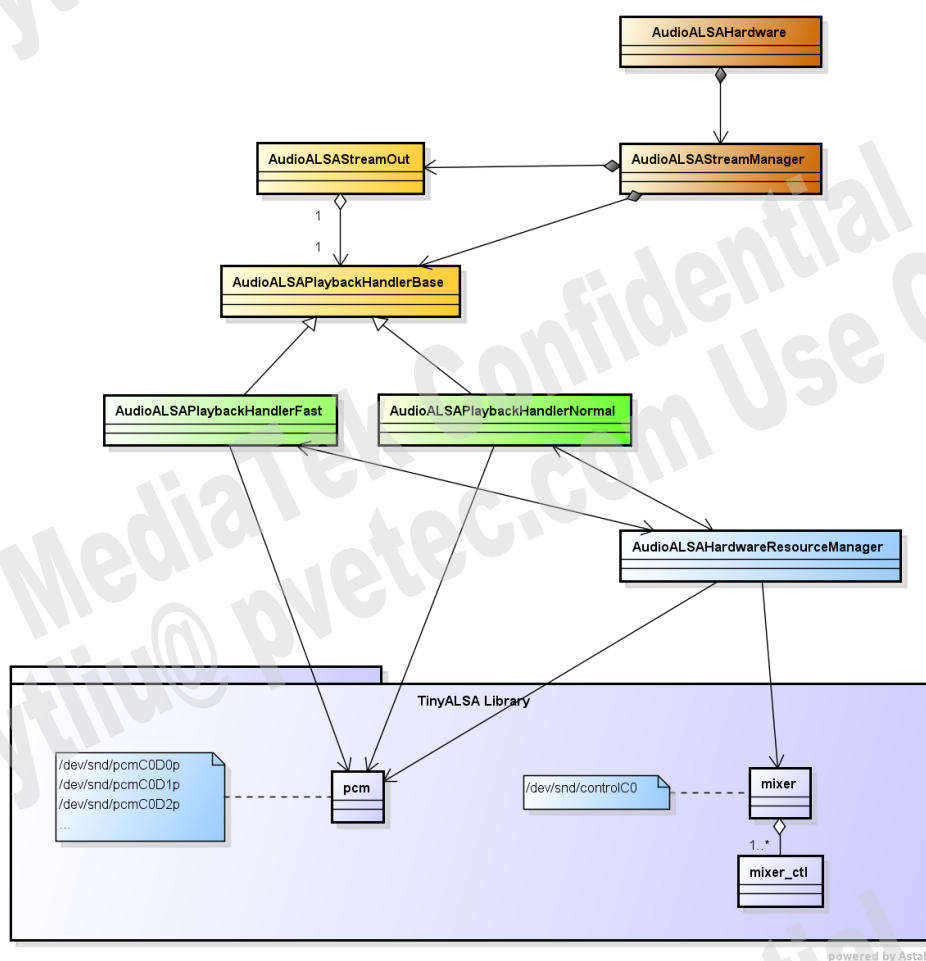


图 2-4. 播放架构

2.1.2 音频 HAL 录制

AudioALSAStreamIn：执行读取/待机/路由操作的类

- 不实现细节本身，而是使用 Capture Handler 绕过打开/关闭/路由/读取操作

- 不同场景使用不同类型的 Capture Handler
- Stream In 将在第一次 read()时调用 StreamManager 中的 createCaptureHandler()来获取 Capture Handler 的指针
- 此外, 在 standby()时, destroyCaptureHandler()

不要在 Capture Handler 中设置硬件寄存器，而是使用 TinyALSA 库来调用

- `pcm_open() / pcm_close()`
 - 用于控制 AFE 硬件路径、内存设置
- `mixer_open() / mixer_close()`
 - 用于获取各种类型的混音器控制，如 `Audio_ADC_1_Switch`、`Audio_ADC_2_Switch`、`Audio_Preamplifier1_Switch`、`Audio_Preamplifier2_Switch`，以控制编解码器驱动程序
- `pcm_read()`
 - 用于将 PCM 数据读取到 SRAM/DRAM 中

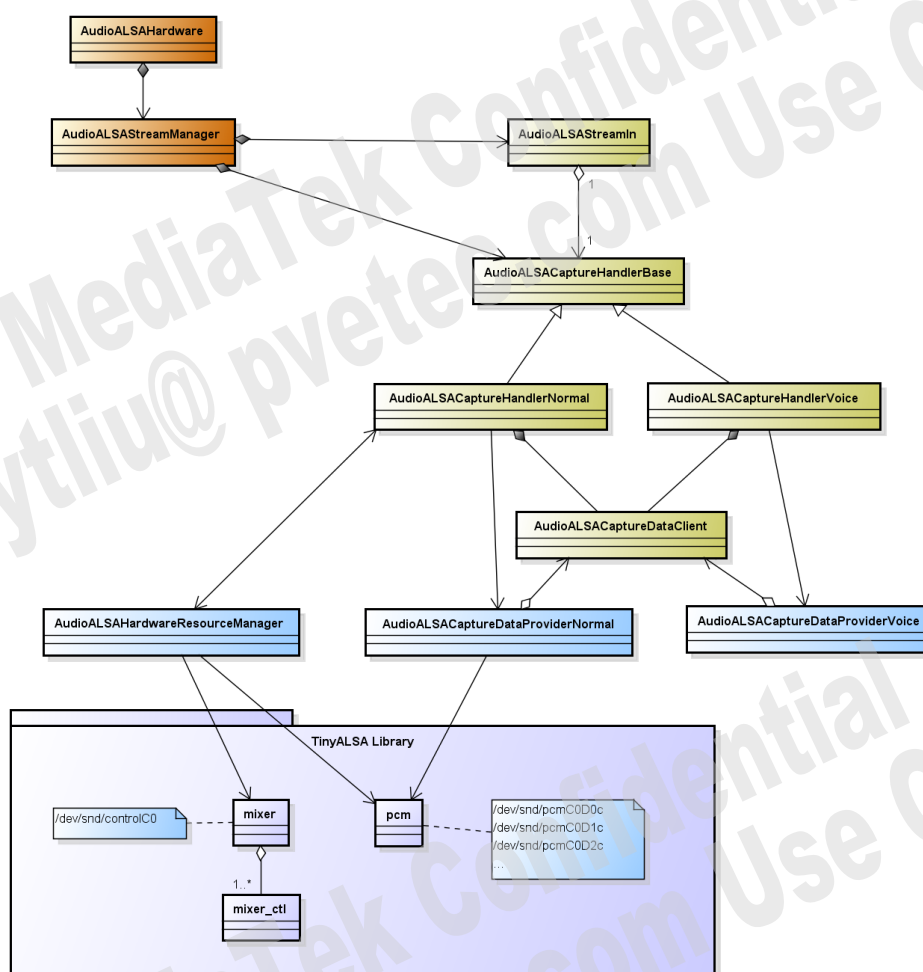


图 2-5. 录制架构

2.2 ALSA 驱动架构概要

高级 Linux 声音架构 (ALSA) 为 Linux 操作系统提供音频和 MIDI 功能。ALSA 具有以下重要功能：

- 高效支持所有类型的音频接口，从消费级声卡到专业多声道音频接口
- 完全模块化的声音驱动程序
- SMP 和线程安全设计
- 用户空间库 (alsa-lib) 简化应用程序编程并提供更高级别的功能
- 支持较旧的开放声音系统 (OSS) API，为大多数 OSS 程序提供二进制兼容性

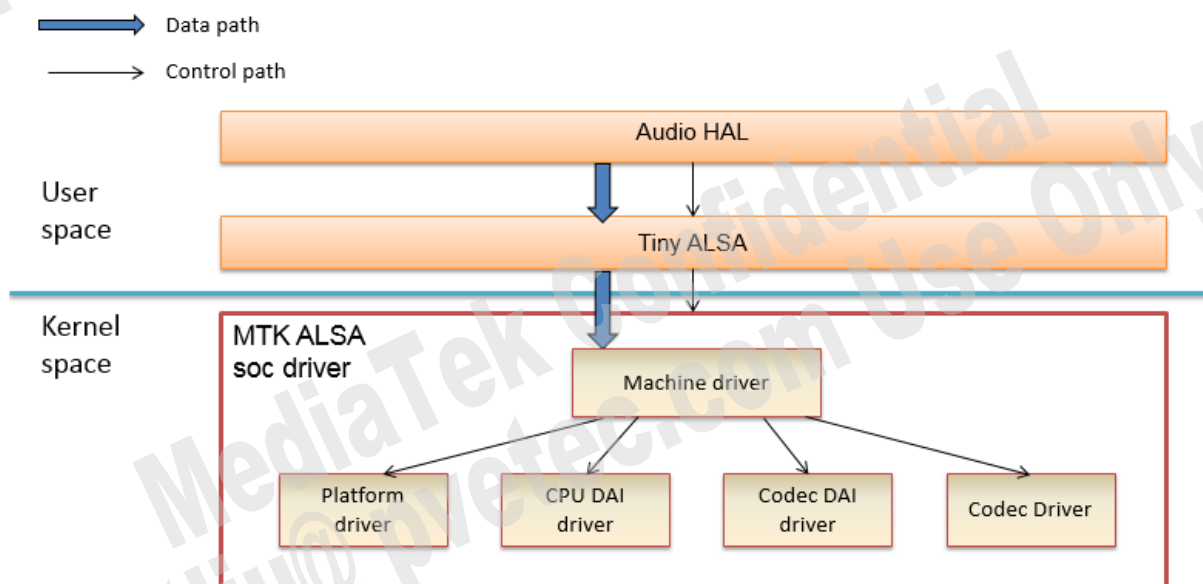


图 2-6. ALSA PCM 接口

2.3 DAPM 及 DPCM 概述

动态音频电源管理（DAPM）旨在允许便携式 Linux 设备始终使用音频子系统的最小电量。它独立于其他内核 PM，因此可以轻松与其他 PM 系统共存。

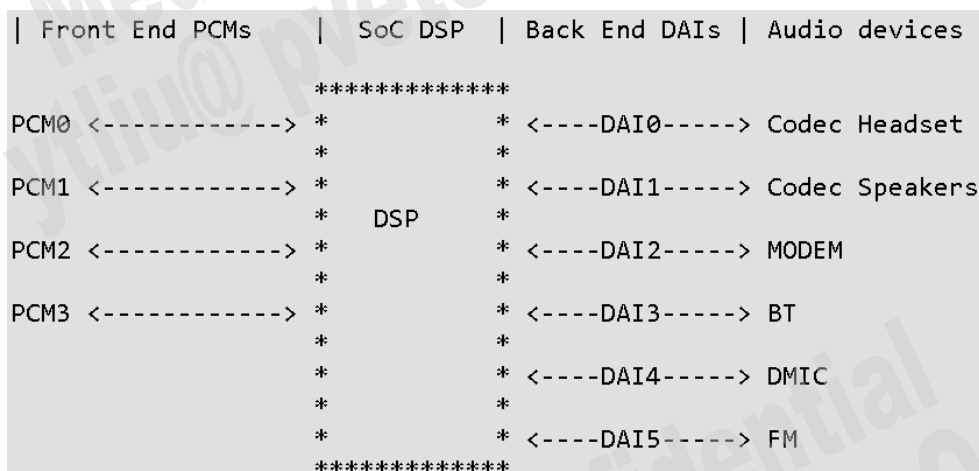
DAPM 对所有用户空间应用程序也完全透明，因为所有电源切换都在 ASoC 核心内完成。用户空间应用程序无需更改代码或重新编译。DAPM 根据设备内的任何音频流（捕获/播放）活动和音频混音器设置做出电源切换决策。

所有 DAPM 电源切换决策都是通过查阅整个机器的音频路由图自动做出的。此图特定于每台机器，由每个音频组件（包括内部编解码器组件）之间的互连组成。以下所有影响电源的音频组件都称为小部件。有关更多详细信息，请参阅：<https://www.kernel.org/doc/html/v6.1/sound/soc/dpcm.html>。

音频驱动基于标准 DAPM 和 DPCM（动态 PCM）架构来控制 pcm 的打开/关闭、电源的开/关、时钟的开/关。

DPCM 将 PCM 分为 Front End/Back End PCM：

- FE PCM: Control of DMA (MEMMEMIF)
- BE PCM: Control of DAI (ADDA, PCM IF, I2S)



更多详细信息可以参考: <https://www.kernel.org/doc/html/v6.1/sound/soc/dpcm.html>。

以下是 MT8668 FE (memif) 和 BE (ADDA/I2S/PCM/Hostless/..) 框图：

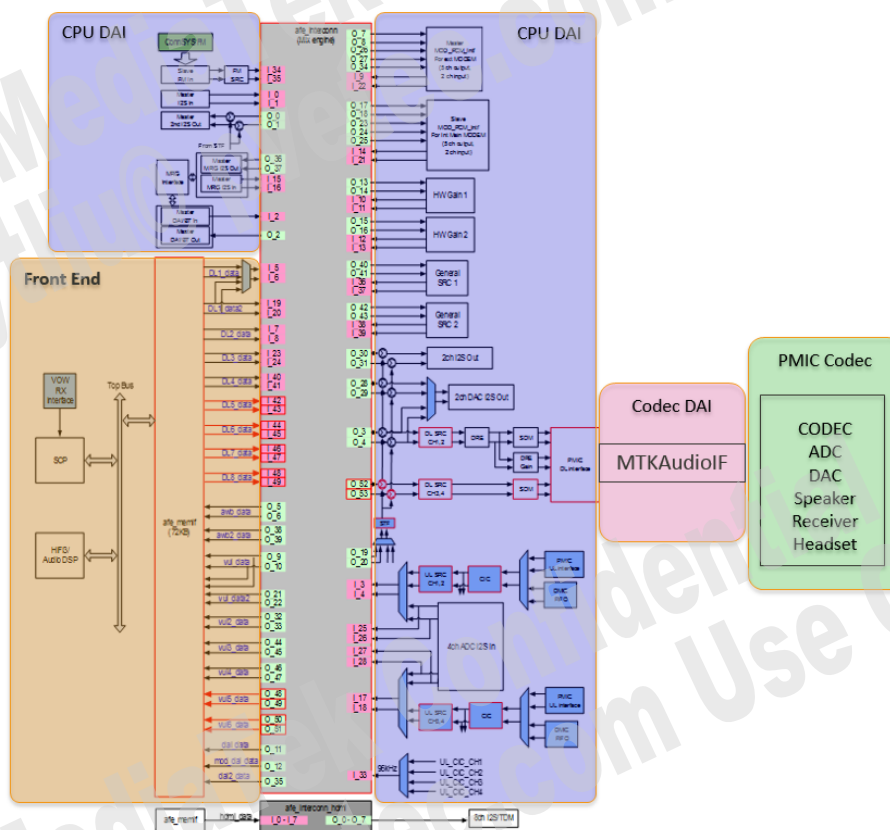


图 2-7. 音频前端和后端框图

3 AAOS 音量控制

根据谷歌的定义，AAOS 的实现是使用硬件功放来控制音量，而不是软件混音器，所以建议使用 smartpa 或者其他硬件增益来调节音量。MTK 在内部 adsp 中提供了调节音量的接口和软件路径，如图 3-1 所示。

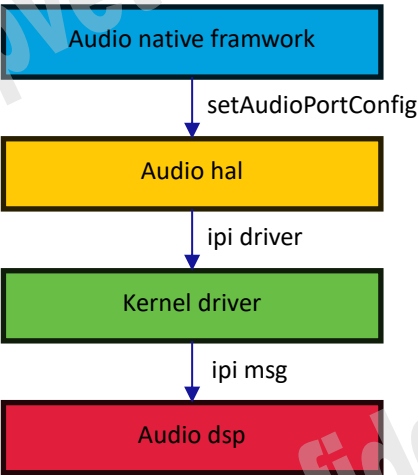


图 3-1. 音频音量控制

3.1 在 XML 文件中配置音量

要配置音量，我们需要修改 car_audio_configuration.xml 和 audio_policy_configuration.xml。

car_audio_configuration.xml 定义音量组，用于管理音频区域内一组设备的音量。对于每个音量组，音量都可以独立控制。产生的增益在相关设备上配置，以供车辆的放大器应用。每个音量组应包含一个或多个具有相关地址的输出设备。地址应与 audio_policy_configuration.xml 中定义的输出设备相对应。

```
<carAudioConfiguration version="3">
  <zones>
    <zone name="primary zone" isPrimary="true" audioZoneId="0" occupantZoneId="0">
      <zoneConfigs>
        <zoneConfig name="primary zone config" isDefault="true">
          <volumeGroups>
            <group>
              <device address="bus0_media_out">
                <context context="music"/>
              </device>
              <device address="bus3_call_ring_out">
                <context context="call_ring"/>
              </device>
              <device address="bus6_notification_out">
                <context context="notification"/>
              </device>
              <device address="bus7_system_sound_out">
                <context context="system_sound"/>
                <context context="emergency"/>
                <context context="safety"/>
                <context context="vehicle_status"/>
                <context context="announcement"/>
              </device>
            </group>
          </volumeGroups>
        </zoneConfig>
      </zoneConfigs>
    </zone>
  </zones>
</carAudioConfiguration>
```

图 3-2. 音频音量组配置

Audio_policy_configuration.xml 定义了音量组增益。每个音量组都有最小、最大和默认增益值，以及基于 audio_policy_configuration.xml 中与音量组关联的设备配置的值的步长。

```
<devicePort tagName="bus0_media_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus0_media_out">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
  </gains>
</devicePort>
<devicePort tagName="bus1_navigation_out" role="sink" type="AUDIO_DEVICE_OUT_BUS"
  address="bus1_navigation_out">
  <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
    samplingRates="48000" channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
  <gains>
    <gain name="" mode="AUDIO_GAIN_MODE_JOINT"
      minValueMB="0" maxValueMB="300" defaultValueMB="300" stepValueMB="30"/>
  </gains>
</devicePort>
```

图 3-3. 设备增益配置

我们提供最小 0 到最大 300 的增益等级，并且增益将通过对数转换函数映射到浮点值 0~1 并应用于数字音频流。

3.2 客制音量命令

音量命令由音频 flinger 通过 setAudioPortConfig 发送，并在 AudioALSAPlaybackHandlerXXXX::setVolume 中处理命令。客户可以修改函数实现以进行自定义，例如调整音量曲线和设置 kcontrol。

4 外部硬件设备

4.1 MT8668 I2S 能力支持

I2S 能力接口描述

- I2S0 – ADC/DAC 接口
 - 适用于任何音频接口
 - 支持**主输入**的标准 I2S
 - 输入能力高达 192kHz/32 位
- I2S1 – ADC/DAC 接口
 - 支持**主输入/主输出**的标准 I2S
 - 最多 1 条数据线支持两个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384k/32 位
- I2S2 – ADC/DAC 接口
 - 支持**主输入/主输出**的标准 I2S
 - 最多 2 条数据线支持四个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- I2S6 – ADC/DAC 接口
 - 支持**主输入/主输出**的标准 I2S
 - 最多 2 条数据线支持 32 个通道
 - 输入能力高达 192kHz/32 位
 - 输出能力高达 384kHz/32 位
- Connsys I2S – ADC 接口
 - 支持标准 I2S **主/从输入**
 - 最多支持 1 条数据线，实现 2 声道
 - 输入能力最高可达 192k/32bit
- 高速 I2S – ADC 接口
 - 支持标准 I2S **从输入**
 - 最多支持 4 条数据线，实现 8 声道
 - 输入能力最高可达 2048k/18bit

表 4-1. IP/Pin 映射表

IP	Pin Name
I2S0	I2SIN0_LRCK I2SIN0_BCK I2S0_MCK I2SIN0_DI
I2S1	I2SIN1_LRCK I2SIN1_BCK I2S1_MCK I2SIN1_DI I2SOUT1_DO
I2S2	I2SIN2_LRCK I2SIN2_BCK I2SIN2_DI I2SOUT2_DO
I2S6	I2SIN6_LRCK I2SIN6_BCK I2SIN6_DI I2SOUT6_DO
Connsys I2S	FMI2SIN_BCK FMI2SIN_LRCK FMI2SIN_MCK FMI2SIN_DI
High Speed I2S	I2SIN_DMA0_BCK I2SIN_DMA0_LRCK I2SIN_DMA0_DI I2SIN_DMA0_DI2 I2SIN_DMA0_DI3 I2SIN_DMA0_DI4

- MCLK 支持 (I2S0/I2S1/I2S2/I2S6)
 - 同步 MCLK 和 I2S BCK
 - 192k 模式下的能力高达 256fs
 - 96k 模式下的能力高达 512fs
 - 48k 模式下的能力高达 1024fs
 - 不支持 192fs 或 384fs 相关

4.2 I2S 连接应用简介

联发科在 HW 功能 (IP: 知识产权) 和 Pad 之间采用高度灵活的设计。

IP 和 Pad 之间的关系可以使用 AUX_FUNC 来选择 Pad 连接到哪个 IP。

表 4-2. GPIO 功能

Aux Func.0	Aux Func.1	Aux Func.2	Aux Func.3	Aux Func.4	Aux Func.5	Aux Func.6
B:GPIO7	-	I0:I2SIN_DMA0_D I3	-	-	-	-
B:GPIO28	-	B0:FMI2SIN_BCK	-	-	-	-
B:GPIO29	-	B0:FMI2SIN_LRCK	-	-	-	-
B:GPIO30	-	O:FMI2SIN_MCK	-	-	-	-
B:GPIO31	-	I0:FMI2SIN_DI	-	-	-	-
B:GPIO32	-	O:I2SIN0_BCK	-	-	-	-
B:GPIO33	-	O:I2SIN0_LRCK	-	-	-	-
B:GPIO34	-	O:I2SIN0_MCK	-	-	-	-
B:GPIO35	-	I0:I2SIN0_DI	-	-	-	-
B:GPIO70	O:I2SIN1_BCK	I0:I2SIN_DMA0_B CK	-	-	-	-
B:GPIO71	O:I2SIN1_LRCK	I0:I2SIN_DMA0_L RCK	-	-	-	-
B:GPIO72	I0:I2SIN1_DI	I0:I2SIN_DMA0_D I	-	-	-	-
B:GPIO73	O:I2SOUT1_DO	I0:I2SIN_DMA0_D I1	-	-	-	-
B:GPIO74	O:I2SIN1_MCK	I0:I2SIN_DMA0_D I2	-	-	-	-
B:GPIO75	O:I2SIN2_BCK	-	-	-	-	-
B:GPIO76	O:I2SIN2_LRCK	-	-	-	-	-
B:GPIO77	I0:I2SIN2_DI	-	-	-	-	-
B:GPIO78	O:I2SOUT2_DO	-	-	-	-	-
B:GPIO79	O:I2SIN2_MCK	-	-	-	-	-
B:GPIO86	-	-	B0:I2SIN6_BCK	-	-	-
B:GPIO87	-	-	O:I2SOUT6_DO	-	-	-

Aux Func.0	Aux Func.1	Aux Func.2	Aux Func.3	Aux Func.4	Aux Func.5	Aux Func.6
B:GPIO88	-	-	O:I2SOUT6_DO1	-	-	-
B:GPIO89	-	-	O:I2SOUT6_DO2	-	-	-
B:GPIO90	-	-	B0:I2SIN6_LRCK	-	-	-
B:GPIO91	-	-	O:I2SOUT6_DO3	-	-	-
B:GPIO97	-	-	I0:I2SIN6_DI	-	-	-
B:GPIO98	-	-	I0:I2SIN6_DI1	-	-	-
B:GPIO99	-	-	I0:I2SIN6_DI2	-	-	-
B:GPIO100	-	-	I0:I2SIN6_DI3	-	-	-
B:GPIO101	-	-	O:I2SIN6_MCK	-	-	-
B:GPIO107	-	O:I2SIN2_BCK	-	-	-	-
B:GPIO108	-	O:I2SIN2_LRCK	-	-	-	-
B:GPIO109	-	I0:I2SIN2_DI	-	-	-	-
B:GPIO110	-	O:I2SOUT2_DO	-	-	-	-
B:GPIO189	-	-	-	I0:I2SIN_DMA0_BCK	-	-
B:GPIO192	-	-	-	I0:I2SIN_DMA0_LRCK	-	-
B:GPIO193	-	-	-	I0:I2SIN_DMA0_DI	-	-
B:GPIO194	-	-	-	I0:I2SIN_DMA0_DI1	-	-
B:GPIO196	-	-	-	I0:I2SIN_DMA0_DI2	-	-
B:GPIO197	-	-	-	I0:I2SIN_DMA0_DI3	-	-

4.2.1 I2S 应用

表 4-3. I2S 接口分配建议

Interface	Direction	Suggestion for I2S IF Assignment
I2S0	Input/	Reserved
I2S1	Input/Output	Passenger BT
I2S2	Input/Output	xCall codec
I2S6	Input/Output	A2B/ External ADSP
Connsys I2S	Input	External FM IC or DP in codec
High Speed I2S	Input	DAB

其他应用，请咨询 MTK ACS 团队以获得最佳 I2S IF 分配。

5 联发科 Aurisys 和开放式 DSP

5.1 概述

人们对音质增强以增加聆听乐趣的需求日益增加。为了丰富使用移动设备的聆听体验，供应商提供的声音处理解决方案的数量不断增加。

Aurisys 的推出是为了促进 MTK 平台声音处理解决方案的开发和使用。Aurisys 是基于 Android 音频框架构建的框架。它包括用于声音处理和调音的标准化接口、集成的 DSP 声音子系统和软件调试接口。本文将简要介绍 Aurisys 的概念。

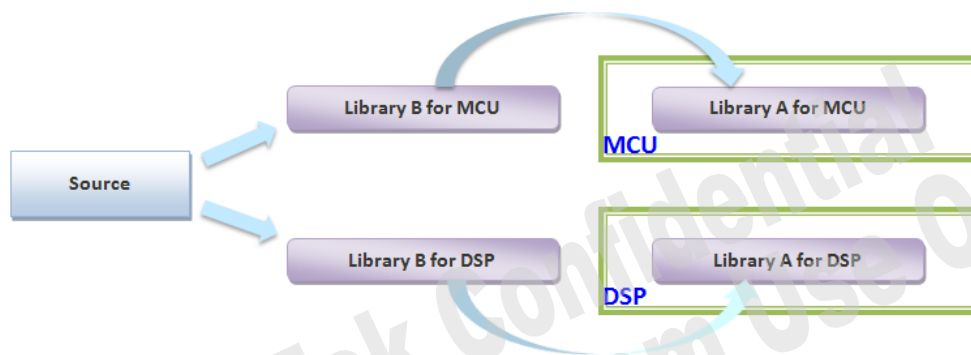


图 5-1. Aurisys 概念

5.1.1 Aurisys 结构

如图 5-2 所示，Aurisys 结构包含场景处理程序、库管理器、集成 DSP 框架、调制解调器/音频硬件子系统和标准化软件接口：

- Aurisys Scene Handler (ARSH)
 - Aurisys Scene Handler 是音频系统和声音处理 IP 之间的中间件，是按场景创建的。例如，Aurisys Playback Handler 是用于播放场景中的播放效果。它提供调用相应 IP 的接口并管理 IP。
- Aurisys Library Manager (ARLM)
 - Aurisys 库管理器维护每个 Aurisys 场景处理程序的库信息。它包含声音算法 IP 的列表和状态。
- Aurisys Software Interface (ARSI)
 - ARSI 是 Aurisys Software Interface 的缩写。它用于与声音增强任务进行交互。为了处理声音，提供了一个可在 MTK 平台之间移植的统一接口。还包括在 PC 工具、APMCU 和 DSP 之间解析和传输参数的接口。该接口设计易于使用，同时足够通用，因此可以轻松将新算法添加到现有框架中。ARSI 和 ARSH 之间的区别在于，ARSI 提供了 ARSH 和声音 IP 之间的接口；而 ARSH 提供了 Android Audio HAL 请求处理声音的接口。

- Modem
 - 调制解调器是指调制解调器 IC。MTK 内部芯片组包含一个 DSP，用于处理语音增强和语音编解码器。在 Aurisys 结构中，我们禁用 modem 内部的语音增强，但在调制解调器 IC 中保留编解码器，以便我们只需在应用程序端集成的开放 DSP 中添加算法即可。
- Audio HW
 - 指处理器与硬件设备之间传输声音数据的接口。这些设备包括扬声器、麦克风、耳机、蓝牙设备、USB 设备等。

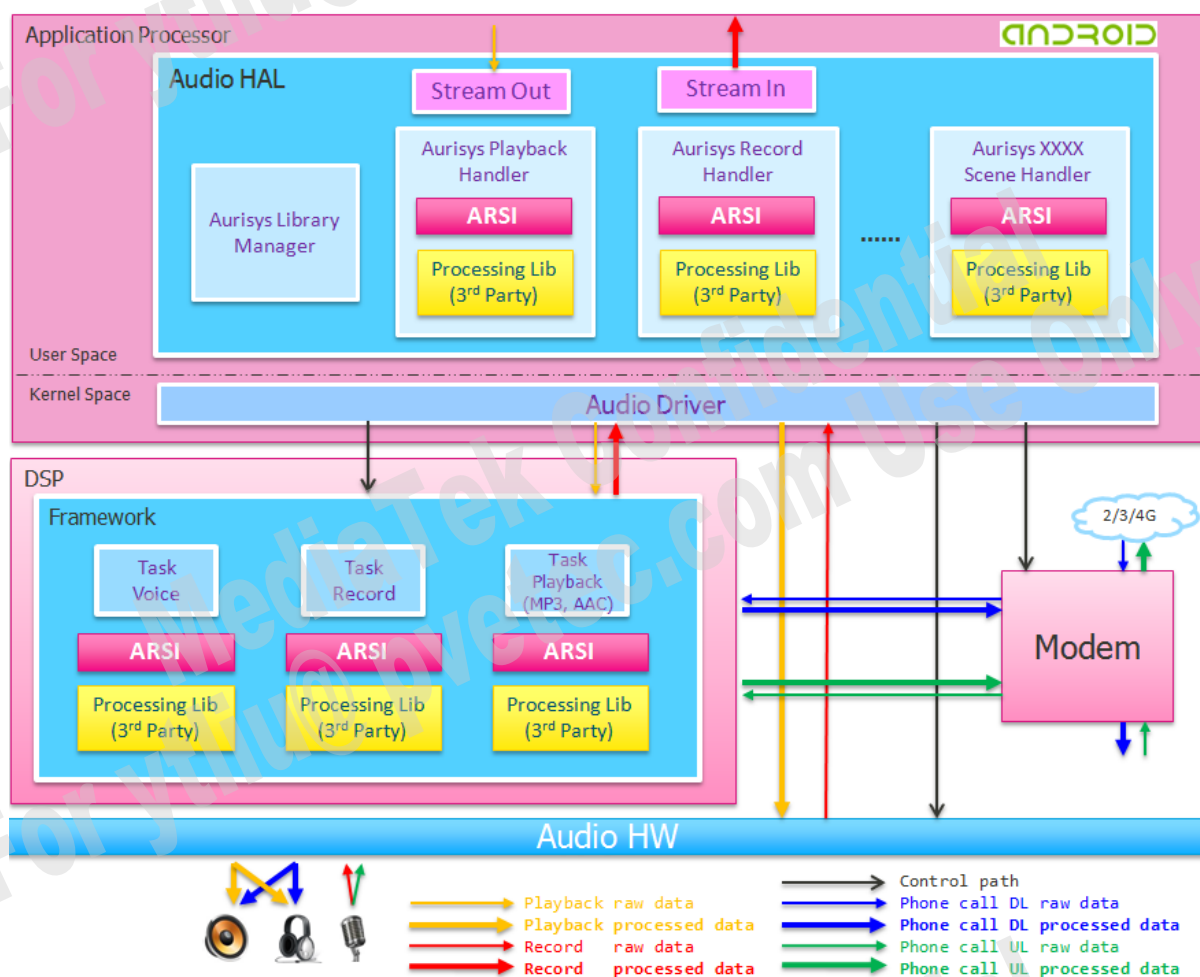


图 5-2. Aurisy 结构

5.2 数据路径客制指南

使用 DSP 进行音频处理和输出时，需要在 DSP 中配置每条音频流的混音路由路径和通道配置信息。公共的 DSP 任务配置提供了基本的参考配置，但在实际场景中，需要对各个 DSP 任务进行一些修改。本节主要介绍 ADSP 相关功能定制所涉及的配置文件修改。

6 Kernel DTS 配置

在 DTS 中，与 DSP 音频流相关的属性配置如下：

```
&snd_audio_dsp {
    mtk-dsp-call-final = <0x0 0xe 0x13 0x12 0x18000>;
    mtk-dsp-hfp-client-rx = <0x5 0xffffffff 0x12 0xffffffff 0x20000>;
    mtk-dsp-anc = <0x5 0xffffffff 0x10 0xffffffff 0x20000>;
    mtk-dsp-extstream1 = <0x5 0xffffffff 0xf 0xffffffff 0x20000>;
    mtk-dsp-extstream2 = <0x5 0xffffffff 0x11 0xffffffff 0x20000>;
    mtk-dsp-sub-playback = <0x1 0x0 0xffffffff 0xffffffff 0x30000>;
    mtk-dsp-playback0 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback1 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback2 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback3 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback4 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback5 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback6 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback7 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback8 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback9 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback10 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback11 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback12 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback13 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback14 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-playback15 = <0x5 0xffffffff 0xffffffff 0xffffffff 0x10000>;
    mtk-dsp-capture-mch = <0x1 0xffffffff 0x18 0xffffffff 0x20000>;
    swdsp-smartpa-process-enable = <0x0>;
};
```

图 6-1. DTS 属性列表

每个 DSP 任务属性节点保存一个五位数组，其中数组顺序代表了该 DSP 任务使用的共享内存的 enable、dl_mem、ul_mem、ref_mem 和大小。在所有属性中，用户需要根据每个 DSP 任务对 memif 的实际使用情况来配置 dl_mem（下行）、ul_mem（上行）、ref_mem（参考信号）。例如，对于 mtk-dsp-sub-playback 任务，使用的 dl 输出 memif 为 DL0，那么对应的 dl_mem 就填充 0x0。大部分任务（比如 playback0）实际上并不使用 hw memif，所以需要配置为 0xffffffff。memif 节点的枚举类型定义在 mt6897-afe-common.h 中。

```

unsigned int virtio_id_memif_index_mapping[MT6881_MEMIF_NUM] = {
    MT6881_MEMIF_DL0,
    MT6881_MEMIF_DL1,
    MT6881_MEMIF_DL2,
    MT6881_MEMIF_DL3,
    MT6881_MEMIF_DL4,
    MT6881_MEMIF_DL5,
    MT6881_MEMIF_DL6,
    MT6881_MEMIF_DL7,
    MT6881_MEMIF_DL8,
    MT6881_MEMIF_DL23,
    MT6881_MEMIF_DL24,
    MT6881_MEMIF_DL25,
    MT6881_MEMIF_DL_24CH,
    MT6881_MEMIF_DL_48CH,
    MT6881_MEMIF_VUL9,
    MT6881_MEMIF_VUL1,
    MT6881_MEMIF_VUL0,
    MT6881_MEMIF_VUL3,
    MT6881_MEMIF_VUL7,
    MT6881_MEMIF_VUL4,
    MT6881_MEMIF_VUL2,
    MT6881_MEMIF_VUL5,
    MT6881_MEMIF_VUL_CM0,
    MT6881_MEMIF_VUL_CM1,
    MT6881_MEMIF_VUL_CM2,
    MT6881_MEMIF_VUL10,
    MT6881_MEMIF_VUL6,
    MT6881_MEMIF_VUL25,
    MT6881_MEMIF_VUL8,
    MT6881_MEMIF_VUL24,
    MT6881_MEMIF_ETDM_IN1,
    MT6881_MEMIF_ETDM_IN2,
    MT6881_MEMIF_ETDM_IN6,
};

```

图 6-2. memif 定义

需要注意的是，在修改了 dts 中对应的 task 使用的节点后，同时也需要修改 hal 中对应的 task 使用的 pcm。Dsp-playback task 对应修改 audio hal 中 AudioDspStreamManager.cpp 中的 openPlaybackTask 函数中的 mPlaybackUIndex 等变量，选用对应 dl 的 playback stream，其中 dsp 输出的 channel 数量等也在该函数处修改，如图 6-3 所示。

```

.../* get card index and pcm index */
.../* todo: pcm should move to platform */
...mStreamCardIndex = AudioALSADeviceParser::getInstance()->GetCardIndexByString(keypcmPlayback11);
...mDspIndex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmPlaybackDsp);

...mPlaybackUIndex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmCapture6);
...mPlaybackIVIndex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmCapture4);
...mPlaybackDLIndex = AudioALSADeviceParser::getInstance()->GetPcmIndexByString(keypcmPlayback11);

```

图 6-3. 音频 HAL PCM Config 配置

DSP 下的各个 task 的架构如下：

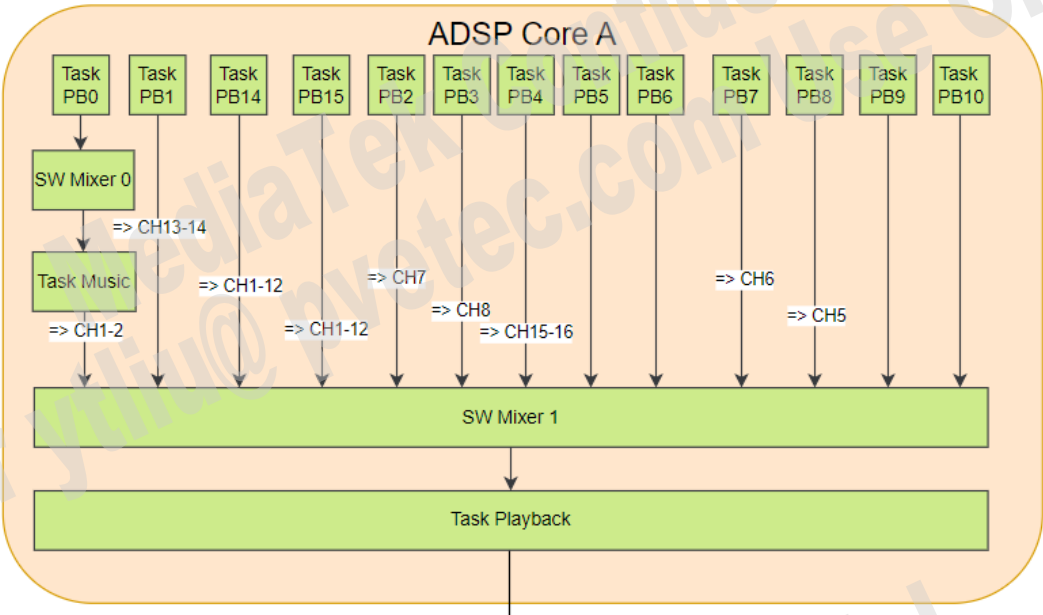


图 6-4. ADSP 架构

6.1 音频路径配置 (AFE HW)

在通过 audio dsp pcm stream 进行播放和录音之前，需要配置对应的 AFE hardware 通路，其配置过程遵循 tinyalsa 的标准流程，通过 tinymix 配置相应的 kcontrol 通路，并 open 和 start 相应的 HW 通路。图 6-5 为配置 DL_MCH2->I2SOUT6 的播放通路。

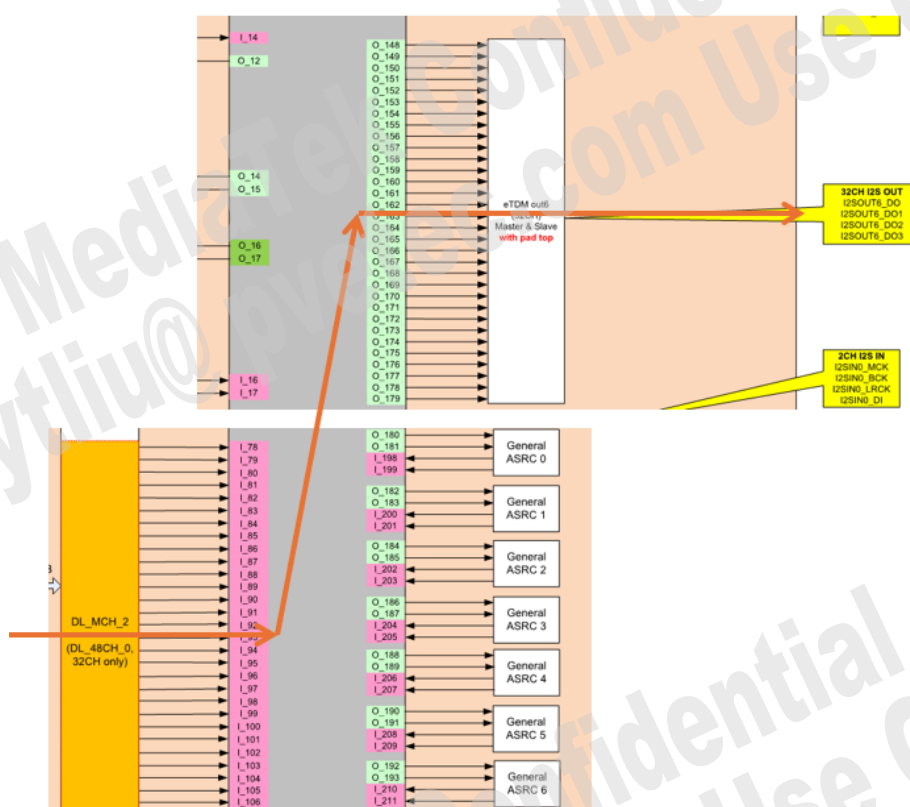


图 6-5. 播放音乐的音频路径

在 audio_device.xml 中相应的配置如下：

connect DL_MCH2 <==> I2SOUT6

```
<path name="DL_M2_32CH_TO_I2SOUT6" value="turnon">
  <kctl name="I2SOUT6_CH1 DL_48CH_CH1" value="1" />
  <kctl name="I2SOUT6_CH2 DL_48CH_CH2" value="1" />
  <kctl name="I2SOUT6_CH3 DL_48CH_CH3" value="1" />
  <kctl name="I2SOUT6_CH4 DL_48CH_CH4" value="1" />
  <kctl name="I2SOUT6_CH5 DL_48CH_CH5" value="1" />
  <kctl name="I2SOUT6_CH6 DL_48CH_CH6" value="1" />
  <kctl name="I2SOUT6_CH7 DL_48CH_CH7" value="1" />
  <kctl name="I2SOUT6_CH8 DL_48CH_CH8" value="1" />
  <kctl name="I2SOUT6_CH9 DL_48CH_CH9" value="1" />
  <kctl name="I2SOUT6_CH10 DL_48CH_CH10" value="1" />
  <kctl name="I2SOUT6_CH11 DL_48CH_CH11" value="1" />
  <kctl name="I2SOUT6_CH12 DL_48CH_CH12" value="1" />
  <kctl name="I2SOUT6_CH13 DL_48CH_CH13" value="1" />
  <kctl name="I2SOUT6_CH14 DL_48CH_CH14" value="1" />
  <kctl name="I2SOUT6_CH15 DL_48CH_CH15" value="1" />
  <kctl name="I2SOUT6_CH16 DL_48CH_CH16" value="1" />
  <kctl name="I2SOUT6_CH17 DL_48CH_CH17" value="1" />
  <kctl name="I2SOUT6_CH18 DL_48CH_CH18" value="1" />
  <kctl name="I2SOUT6_CH19 DL_48CH_CH19" value="1" />
  <kctl name="I2SOUT6_CH20 DL_48CH_CH20" value="1" />
  <kctl name="I2SOUT6_CH21 DL_48CH_CH21" value="1" />
  <kctl name="I2SOUT6_CH22 DL_48CH_CH22" value="1" />
  <kctl name="I2SOUT6_CH23 DL_48CH_CH23" value="1" />
  <kctl name="I2SOUT6_CH24 DL_48CH_CH24" value="1" />
  <kctl name="I2SOUT6_CH25 DL_48CH_CH25" value="1" />
  <kctl name="I2SOUT6_CH26 DL_48CH_CH26" value="1" />
  <kctl name="I2SOUT6_CH27 DL_48CH_CH27" value="1" />
  <kctl name="I2SOUT6_CH28 DL_48CH_CH28" value="1" />
  <kctl name="I2SOUT6_CH29 DL_48CH_CH29" value="1" />
  <kctl name="I2SOUT6_CH30 DL_48CH_CH30" value="1" />
  <kctl name="I2SOUT6_CH31 DL_48CH_CH31" value="1" />
  <kctl name="I2SOUT6_CH32 DL_48CH_CH32" value="1" />
</path>
```

disconnect DL_MCH2 <==> I2SOUT6

```
<path name="DLW2_32CH_T0_I2SOUT6" value="turnoff">
  <kctl name="I2SOUT6_CH1 DL_48CH_CH1" value="0" />
  <kctl name="I2SOUT6_CH2 DL_48CH_CH2" value="0" />
  <kctl name="I2SOUT6_CH3 DL_48CH_CH3" value="0" />
  <kctl name="I2SOUT6_CH4 DL_48CH_CH4" value="0" />
  <kctl name="I2SOUT6_CH5 DL_48CH_CH5" value="0" />
  <kctl name="I2SOUT6_CH6 DL_48CH_CH6" value="0" />
  <kctl name="I2SOUT6_CH7 DL_48CH_CH7" value="0" />
  <kctl name="I2SOUT6_CH8 DL_48CH_CH8" value="0" />
  <kctl name="I2SOUT6_CH9 DL_48CH_CH9" value="0" />
  <kctl name="I2SOUT6_CH10 DL_48CH_CH10" value="0" />
  <kctl name="I2SOUT6_CH11 DL_48CH_CH11" value="0" />
  <kctl name="I2SOUT6_CH12 DL_48CH_CH12" value="0" />
  <kctl name="I2SOUT6_CH13 DL_48CH_CH13" value="0" />
  <kctl name="I2SOUT6_CH14 DL_48CH_CH14" value="0" />
  <kctl name="I2SOUT6_CH15 DL_48CH_CH15" value="0" />
  <kctl name="I2SOUT6_CH16 DL_48CH_CH16" value="0" />
  <kctl name="I2SOUT6_CH17 DL_48CH_CH17" value="0" />
  <kctl name="I2SOUT6_CH18 DL_48CH_CH18" value="0" />
  <kctl name="I2SOUT6_CH19 DL_48CH_CH19" value="0" />
  <kctl name="I2SOUT6_CH20 DL_48CH_CH20" value="0" />
  <kctl name="I2SOUT6_CH21 DL_48CH_CH21" value="0" />
  <kctl name="I2SOUT6_CH22 DL_48CH_CH22" value="0" />
  <kctl name="I2SOUT6_CH23 DL_48CH_CH23" value="0" />
  <kctl name="I2SOUT6_CH24 DL_48CH_CH24" value="0" />
  <kctl name="I2SOUT6_CH25 DL_48CH_CH25" value="0" />
  <kctl name="I2SOUT6_CH26 DL_48CH_CH26" value="0" />
  <kctl name="I2SOUT6_CH27 DL_48CH_CH27" value="0" />
  <kctl name="I2SOUT6_CH28 DL_48CH_CH28" value="0" />
  <kctl name="I2SOUT6_CH29 DL_48CH_CH29" value="0" />
  <kctl name="I2SOUT6_CH30 DL_48CH_CH30" value="0" />
  <kctl name="I2SOUT6_CH31 DL_48CH_CH31" value="0" />
  <kctl name="I2SOUT6_CH32 DL_48CH_CH32" value="0" />
</path>
```

图 6-6. 将 Playback_3 传输到 PMIC

6.2 音频路径配置 (INT ADSP)

Auto adsp 中会存在多个 task (bus out)，且每个 task 根据需求及实际场景不同，需要配置不同的算法处理参数，例如 channel in，channel out 及是否 bypass 算法等。同时，一般需要将各路 bus 的 channel 映射到最终输出的 TDM 格式中对应的 channel 位置，即需要配置 channel map 信息，故提供 audio_dsp_config.xml 配置修改 auto dsp task 的各项属性及 mix path。一般的 adsp 中的 task 架构如图：

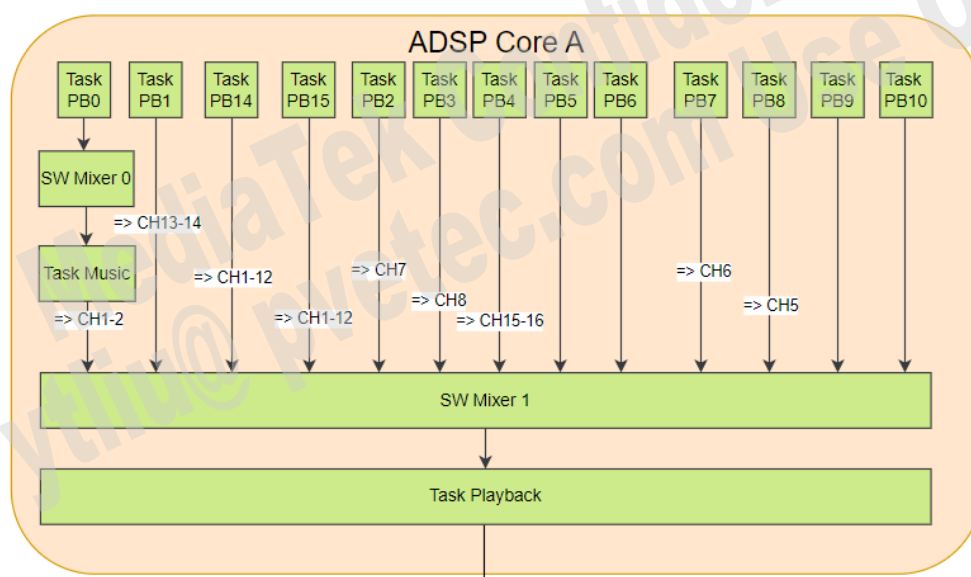


图 6-7. ADSP 架构

6.2.1 XML 配置指令

配置 XML 路径：

- xml 在设备上的 path 如下：vendor/etc/audio_dsp_config.xml
- 在 code 中 path 如下：device/<project_name>/ audio_dsp_config.xml

6.2.2 mixer_target 和 mixer_source

XML 中将每个 dsp task 根据其在 dsp 中的 mixer 的位置分为了 sw_mixer_target 和 sw_mixer_source。对于 Task Music，其即是 sw_mixer_target，也是 sw_mixer_source。

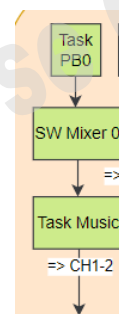
- sw_mixer_target
对应于 sw_mixer 的输出端，如 Task Music, Task Playback, Task SubPlayback
- sw_mixer_source
对应于 sw_mixer 的输入端，如 Task PB0 (name="sw_mixer_playback0")

task 作为 target 或者 source 是不可配置的，其角色是预定好的，但是可以通过 XML 配置 sw_mixer_source mix 到期望的 target 中。对应的修改方法是在 XML 中，将对应 name 的 sw_mixer_source 节点写到对应的 sw_mixer_target 的节点下，如下例子表示 task PB0 (sw_mixer_playback0) 将会 mix 到 task music(sw_mixer_music)中。

```

Ex1:
    <sw_mixer_target name="sw_mixer_music" ch_in="2"
ch_out="16" aurisys_on="1">
    <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2"
aurisys_on="0"
                ch_map="0x5555,0xaaaa"/>
    </sw_mixer_source>
    </sw_mixer_target>

```



目前可以配置的 sw_mixer_source 有: sw_mixer_playback0 ~ sw_mixer_playback15, sw_mixer_FM_ADSP, sw_mixer_hfp_client_rx, sw_mixer_anc, sw_mixer_extstream1, sw_mixer_extstream2, sw_mixer_ktv.

6.2.3 任务属性

在每个 dsp task, 由于场景需要, 会配置不同的算法输入输出 channel 及 channel map 信息。在 XML 中, 可以为每个 task 分别配置下列属性:

- for sw_mixer_target
 - ch_in: Task 的算法输入 channel
 - ch_out: Task 的算法输出 channel
 - aurisys_on: 是否开启 Task 算法, 仅在 ch_in = ch_out 时可配置成 0

例如, 在下面例子中, task music 的算法输入 channel 是 2, 算法处理后的输出 channel 是 16, 算法非 bypass。

```

<sw_mixer_target name="sw_mixer_music" ch_in="2" ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
            ch_map="0x5555,0xaaaa"/>
  </sw_mixer_source>
</sw_mixer_target>

```

- 对于 sw_mixer_source, 每个 profile 有如下属性
 - ch_in: Task 的算法输入 channel
 - ch_out: Task 的算法输出 channel
 - aurisys_on: 是否开启 Task 算法, 仅对 pb0~pb15 在 ch_in = ch_out 时配置成 0 可生效
 - ch_map: source 在向 sw_mixer_target 映射时候的 channel map 信息, 是一个数组, 数组中的 index 表示 source 的 channel index, 对应的值是一个 16bit 的 value, 每一个 bit 表示是否向 target 对应 channel 映射。如 ch_map="0x5555,0xaaaa", ch_map[0] = 0x5555, 表示 source 的 channel 1 映射数值为 0x5555, 对应 2 进制为 0101010101010101, 则向 target 的 1,3,5,7,9,11,13,15 channel 映射。ch_map[1] = 0xaaaa, 对应二进制位 1010101010101010, source 的 channel 2 向 target 的 2,4,6,8,10,12,14,16 映射。

对 source task，一般其输入 channel 是由数据原始端给出的，如 audioflinger 送下来，可能会是任意 channel 数量，所以在 source task 中，XML 可配置支持不同的 ch_in 的多种 profile，可以根据不同的 ch_in 选择不同的 ch_out, ch_map 等信息。例如，如下 task pb14 具有两个 profile，当进入 adsp 给到 task pb14 的 channel 是 2ch，则使用第一个 profile，如果是 12ch，则使用第二个。

```
<sw_mixer_source name="sw_mixer_playback14">
  <profile ch_in="2" ch_out="2" aurisys_on="0"
    ch_map="0x5555,0xaaaa"/>
  <profile ch_in="12" ch_out="12" aurisys_on="0"
    ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800,0x1000,0x2000,0x4000,0x8000"/>
</sw_mixer_source>
```

对于 music task，由于其即是 source，也是 target，则会取其 target 属性中的 ch_in 属性，在 source 中找到对应的 profile 作为其配置。

6.2.4 配置示例

如下是一个 task pb0 配置示例：

```
<sw_mixer_target name="sw_mixer_music" ch_in="8" ch_out="12" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x4,0x8"/>
    <profile ch_in="4" ch_out="4" aurisys_on="0"
      ch_map="0x1,0x2,0x4,0x8 "/>
  </sw_mixer_source>
</sw_mixer_target>

<sw_mixer_target name="sw_mixer_playback" ch_in="16" ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_music">
    <profile ch_in="8" ch_out="12" aurisys_on="1"
      ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800
"/>
  </sw_mixer_source>
</sw_mixer_target>
```

对应的数据通路流程如图 6-8：

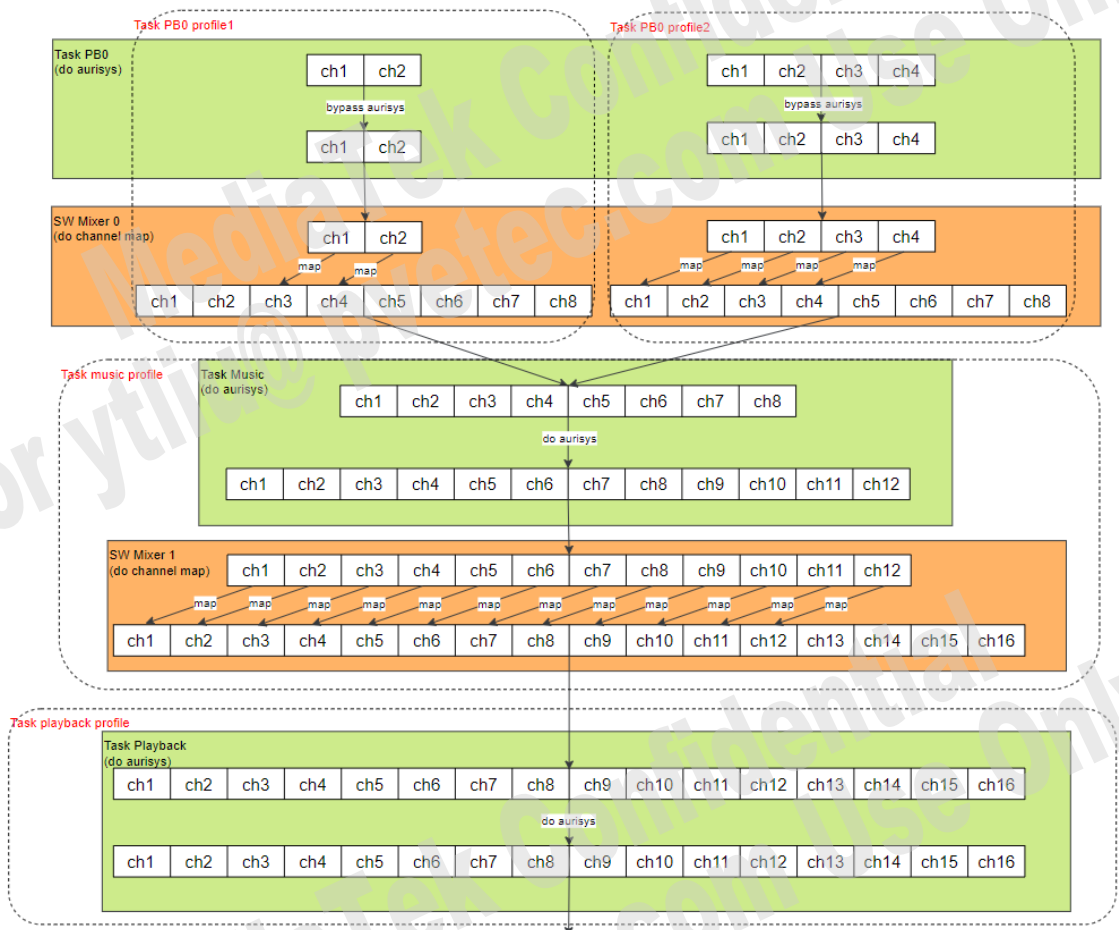


图 6-8. DSP 任务配置

7 音频 HAL 软件混音器

Audio HAL SW Mixer 的作用和 ADSP 内部的 SW mixer 一样，它通过在音频 HAL 中执行通道映射来混合来自多个播放总线的数据，然后通过一个 PCM device node 播放出去。

7.1 音频 HAL 软件混音器代码架构

Audio HAL SW Mixer 功能需求如下：支持多个 playback bus 共用一个 PCM 硬件节点进行输出，允许多个音频源同时工作，并在控制 and 数据流程逻辑上互不干扰。同时，系统可支持多组混音器，且各组之间独立运行，互不影响。

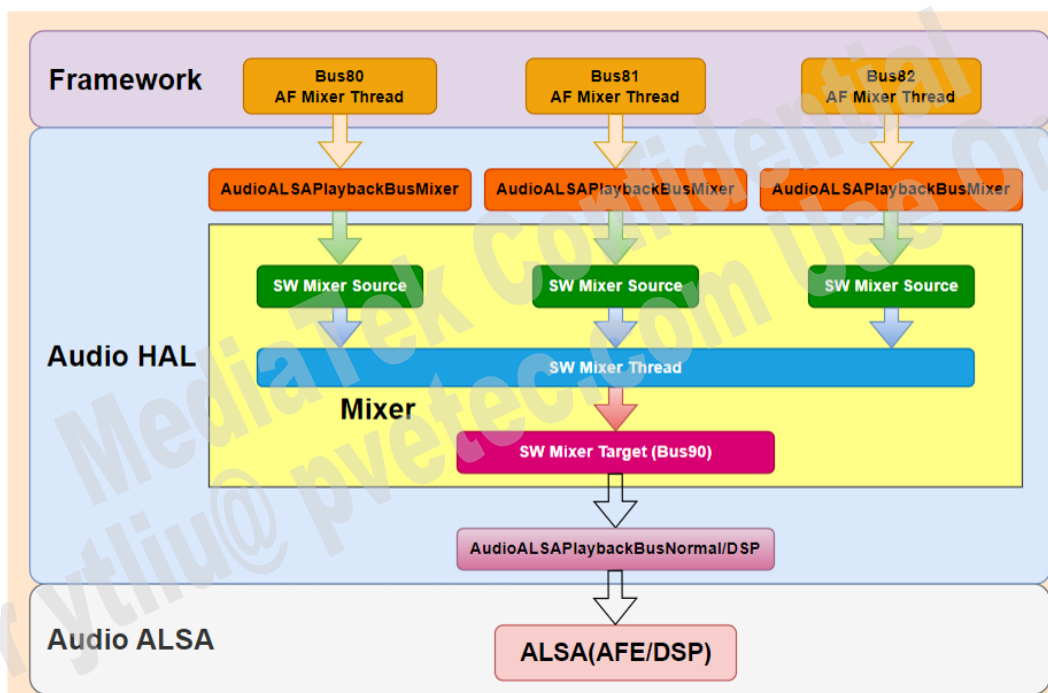


图 7-1. 音频 HAL 软件混音器软件架构

核心文件说明如下：

- AudioALSAPlaybackBusMixer.cpp
 - 该文件继承自 playbackHandlerBase，用于实现 Source 的 PlaybackHandler 功能；
 - 类中包含全局变量 TargetList 和 TargetLock，用于统一管理多组 Mixer 的 Target 资源，Source 可通过 MixerId 查询对应的 Target Bus，并获取 Target 实体。
- audio_sw_mixer_auto.c

- 负责混音功能的实现，包含混音操作的主要逻辑（如 Source/Target 的 attach、write 以及混音线程的实现等）。
- audio_process_utility.c
 - 实现混音及 Channel Map 的核心功能，主要负责单个 Source 的混音及其 Channel Map 操作。
 - 与 audio_sw_mixer_auto.c 配合，实现多路混音功能。
- auto_policy_config.xml
 - 用于定制 Mixer 的混音通路策略，通过配置文件灵活调整混音流程。

7.2 音频 HAL 软件混音器 XML 配置指南

如果客户有定制化需求，可以通过修改 XML 配置文件来实现，主要涉及 audio_policy_configuration.xml 和 auto_policy_config.xml 两个文件：

- audio_policy_configuration.xml
 - 文件名称可能因不同项目而有所差异。
 - 该文件用于声明 Mixer 的 Source Bus 到 AudioFramework，确保 APP 能够正常访问和播放音频。
- auto_policy_config.xml
 - 文件名称可能因不同项目而有所差异。
 - 该文件由 AudioHAL 读取，用于定制 HAL 层的软件混音器（SW Mixer）。
 - 可在此文件中配置 SW Mixer 的 Source Bus、Target Bus 以及 Channel Map，实现灵活的混音通路定制。

7.2.1 先明确相关应用场景的实际需求

例如：某应用场景需要在 HAL 层将 BUS80、BUS81 和 BUS82 三路音频进行混音，并将混音后的音频输出到 BUS90。

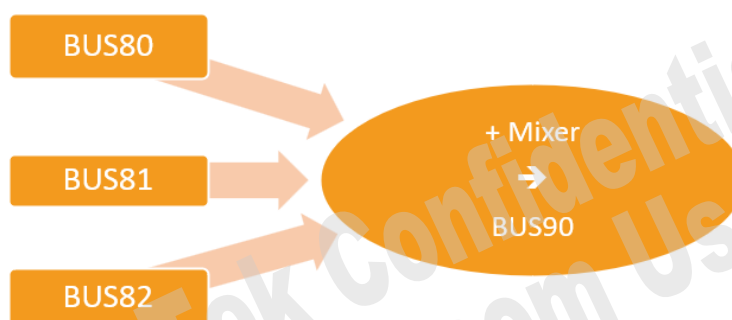


图 7-2. 音频 HAL 软件混音器示意图

7.2.2 声明 BUS80/81/82 给 AudioPolicyManager 及 audioFlinger

- 配置 xml: audio_policy_configuration.xml

- 在 attachedDevices 标签中添加:

```

46      <!-- Primary Audio HAL -->
47      <module name="primary" halVersion="3.0">
48          <attachedDevices>
49              <!-- One bus per context -->
50              <item>bus0_media_out</item>
51              <item>bus1_navigation_out</item>
52              <item>bus2_voice_command_out</item>
53              <item>bus3_call_ring_out</item>
69              <item>Telephony Tx</item>
70              <item>bus220_btsco_in</item>
71              <item>bus80_playback_out</item>
72              <item>bus81_playback_out</item>
73              <item>bus82_playback_out</item>
74          </attachedDevices>

```

- 在 mixPorts 标签中增加对应的 mixport 的配置, 并将 role 设置为 source。此配置主要用于在 AudioFlinger 中创建混音线程:

```

213      <!-- Bus 80/81/82 For Audio HAL Mixer -->
214      <mixPort name="mixport_bus80_playback_out" role="source">
215          <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
216              samplingRates="48000"
217              channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
218      </mixPort>
219
220      <mixPort name="mixport_bus81_playback_out" role="source">
221          <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
222              samplingRates="48000"
223              channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
224      </mixPort>
225
226      <mixPort name="mixport_bus82_playback_out" role="source">
227          <profile name="" format="AUDIO_FORMAT_PCM_32_BIT"
228              samplingRates="48000"
229              channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
230      </mixPort>
231
232      </mixPorts>
233

```

- 在 devicePorts 标签中, 声明 Audio HAL 支持的 3 个 BUS。
- 在 routes 标签中, 将 mixPort 与 devicePort 进行绑定:

```

474      <!-- Bus 80/81/82 For Audio HAL Mixer -->
475      <route type="mix" sink="bus80_playback_out" sources="mixport_bus80_playback_out"/>
476      <route type="mix" sink="bus81_playback_out" sources="mixport_bus81_playback_out"/>
477      <route type="mix" sink="bus82_playback_out" sources="mixport_bus82_playback_out"/>
478
479      </routes>

```

7.2.3 配置 Audio HAL 的 Mixer 通路的路由并配置其为 Target

- 配置 xml: auto_policy_config.xml

- 在 bus_stream_maps 标签中，配置 Target 的 bus 设置。

```
<!-- HAL SW Mixer Target Bus -->
<bus_stream bus_num="90" hal_buffer_frames="1024">
  <pcm_profile id="DL" pcm_stream="DSP_Playback_Playback13" period_frames="1024" period_cnt="4"/>
</bus_stream>
```

注：Target 可以设置为通过 AFE 或 ADSP，请根据实际需求进行设置。

- 在 hal_sw_mixer_routes 标签中，配置 mixer 的路由通路。

```
<hal_sw_mixer_routes>
  <sw_mixer_target address="bus90_internal_use" ch_in="8" ch_out="8" aurisys_on="0" type="AUDIO_DEVICE_OUT_BUS">
    <sw_mixer_source address="bus80_playback_out" >
      <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x1,0x2"/>
    </sw_mixer_source>

    <sw_mixer_source address="bus81_playback_out" >
      <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x4,0x8"/>
    </sw_mixer_source>

    <sw_mixer_source address="bus82_playback_out" >
      <profile ch_in="2" ch_out="2" aurisys_on="0" ch_map="0x10,0x20"/>
    </sw_mixer_source>
  </sw_mixer_target>
</hal_sw_mixer_routes>
```

注意：

- 配置格式是以 Target 标签为父节点，Source 为子节点。
- 当前作为 Source 和 Target 的 Bus 尚未调试 Aurisys 功能，因此 aurisys_on 字段需统一设置为 0。
- 同一配置中 ch_in 与 ch_out 需要保持一致（目前 aurisys 不支持 ch_in 与 ch_out 不一致的情况）
 - Target 中的 ch_out 指定最终混音输出的声道数
 - Source 的 profile 以 ch_in 作为选择条件。Source 可有多种 profile，每个 profile 的 ch_in 标识该 source 的声道数
- Source profile 中的 ch_map 配置
 - ch_map 用于配置混音的声道映射，其数组长度等于 ch_in 的值。
 - ch_map 数组的下标（index）对应 Source 的声道顺序，例如 ch_map[0] 表示 Source 的第 1 声道。
 - ch_map 的每个值表示 Target 的声道使能情况，每一位对应 Target 的一个声道（从低位到高位，依次为 Target 的声道 1~N）。

例如，ch_in = 2ch 时，ch_map 设置为 0x1, 0x2，则：

0x1 (0b0000 0001) 表示 Source 的第 1 声道混音到 Target 的第 1 声道。

0x2 (0b0000 0010) 表示 Source 的第 2 声道混音到 Target 的第 2 声道。

举例说明：

- Source 是 2 声道，Target 是 8 声道的情况，如果 ch_map="0x0f,0xf0" 的解析：

- 0x0f (0b0000 1111) : Source 的第 1 声道混音到 Target 的第 1、2、3、4 声道;
 - 0xf0 (0b1111 0000) : Source 的第 2 声道混音到 Target 的第 5、6、7、8 声道。
2. Source 是 2 声道, Target 是 8 声道的情况, 如果 ch_map="0xff,0xff" 的解析:
- 0xff (0b1111 1111) : Source 的第 1 声道混音到 Target 的全部 8 个声道;
 - 0xff (0b1111 1111) : Source 的第 2 声道同样混音到 Target 的全部 8 个声道。

如果 Target 的每个声道都配置了 Source 的 ch1 和 ch2, 则 Target 的每一个声道数据最终都是由 Source 的 ch1 和 ch2 混音生成的。

8 添加音频编 Codec 驱动

在 MT8668 公版中，集成了 ES8311 和 AK7709 两颗 codec，客户可以参考这两颗 codec 进行客制化芯片的 bring up。公版 codec 的具体连接方式为：AK7709 外接在 I2SOUT6/I2SIN6 上，而 ES8311 接在 I2SOUT2/I2SIN2 上。以下是以 ES8311 codec 驱动为例的相关改动说明：

1. 将 ES8311 的源代码添加到 sound/soc/codecs 目录中。同时，在源代码中增加 BUILD.bazel 文件。

```
├── BUILD.bazel
├── es8311.c
└── es8311.h

1 directory, 3 files
```

BUILD.bazel 内容如下：

```
load("//build/bazel_mgk_rules:mgk_ddk_ko.bzl", "define_mgk_ddk_ko")
load("//build/kernel/kleaf:kernel.bzl", "ddk_headers")
load("@mgk_info//:kernel_version.bzl", "kernel_version",)

package(
    default_visibility = [
        "//visibility:public",
    ],
)

# add ddk_headers to export headers for other modules to use in the future
ddk_headers(
    name = "ddk_public_headers",
    includes = ["."],
    hdrs = glob(["**/*.h"]), # export all headers for other modules to use
)

filegroup(
    name = "ddk_kconfigs",
    srcs = glob([
        "**/Kconfig",
    ]),
)

filegroup(
    name = "srcs",
    srcs = glob([
        "**/Makefile",
        "**/Kbuild",
        "**/*.c",
        "**/*.h",
    ]),
)

define_mgk_ddk_ko(
    name = "snd-soc-es8311",
    srcs = [
        "es8311.c",
    ],
    hdrs = glob(["**/*.h"]),
    includes = ["."],
)
```

2. 在 dts 中添加 es8311 设备节点。

```
&i2c2 {
    status = "okay";
    clock-frequency = <400000>;
    es8311@18 {
        compatible = "everest,es8311";
        #sound-dai-cells = <0>;
        reg = <0x18>;
        status = "okay";
    };
};
```

3. 在 ko table 中添加 snd-soc-es8311.ko。

```
/device/mediatekproject/<project_name>/ko_order_table.csv
```

```
snd-soc-es8311.ko,../../kernel_device_modules-6.12/sound/soc/codecs/es8311/snd-soc-es8311.ko,vendor,Y,N,user/userdebug/eng
```

4. 在 machine driver 中适配 ALSA DAI link:

```
/sound/soc/mediatek/mt6881/mt6881-mt6368.c
```

```
SND_SOC_DAILINK_DEFS(i2sin2,
    DAILINK_COMP_ARRAY(COMP_CPU("I2SIN2")),
    DAILINK_COMP_ARRAY(COMP_CODEC("es8311.2-0018",
        "ES8311 HiFi")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));

SND_SOC_DAILINK_DEFS(i2sout2,
    DAILINK_COMP_ARRAY(COMP_CPU("I2SOUT2")),
    DAILINK_COMP_ARRAY(COMP_CODEC("es8311.2-0018",
        "ES8311 HiFi")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
```

其中，在 COMP_CODEC 中，name 填写<驱动名>.<总线编号>.<地址>; dai_name 填写 codec driver 中的 dai name，也就是 snd_soc_dai_driver 结构体中的 name 字段。

9 总结

本章主要介绍 Aurisys 音效框架，方便用户在适当位置整合所需算法。同时，面对 bus 场景音频通路的客制化需求，介绍了 ADSP 中音频 mix 和 channel map 的配置方法。

除了上述功能外，还可实现其他应用。但考虑到 DSP 资源有限，并发情况应谨慎设计。请访问联发科网站和客户支持网站 (MediaTek On-Line, MOL) 以获取更多信息。

附录 MTK MOL

MTK 提供了一个名为 MTK 在线的论坛，用于分享常见问题、电子课程和重要公告：<http://online.mediatek.com>。

您可以输入关键字来搜索相关的常见问题和电子课程；可以浏览文档树以进入相关的音频常见问题和电子课程。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。